

AI HOME SHIELD

Final End-to-End System Architecture

Hardware + Edge AI + Network Security + Agentic Response + SIH-ready Predictive Extension

Architecture Reference v1.0 - Frozen baseline for implementation, hardware integration, testing, demos, and future productization.

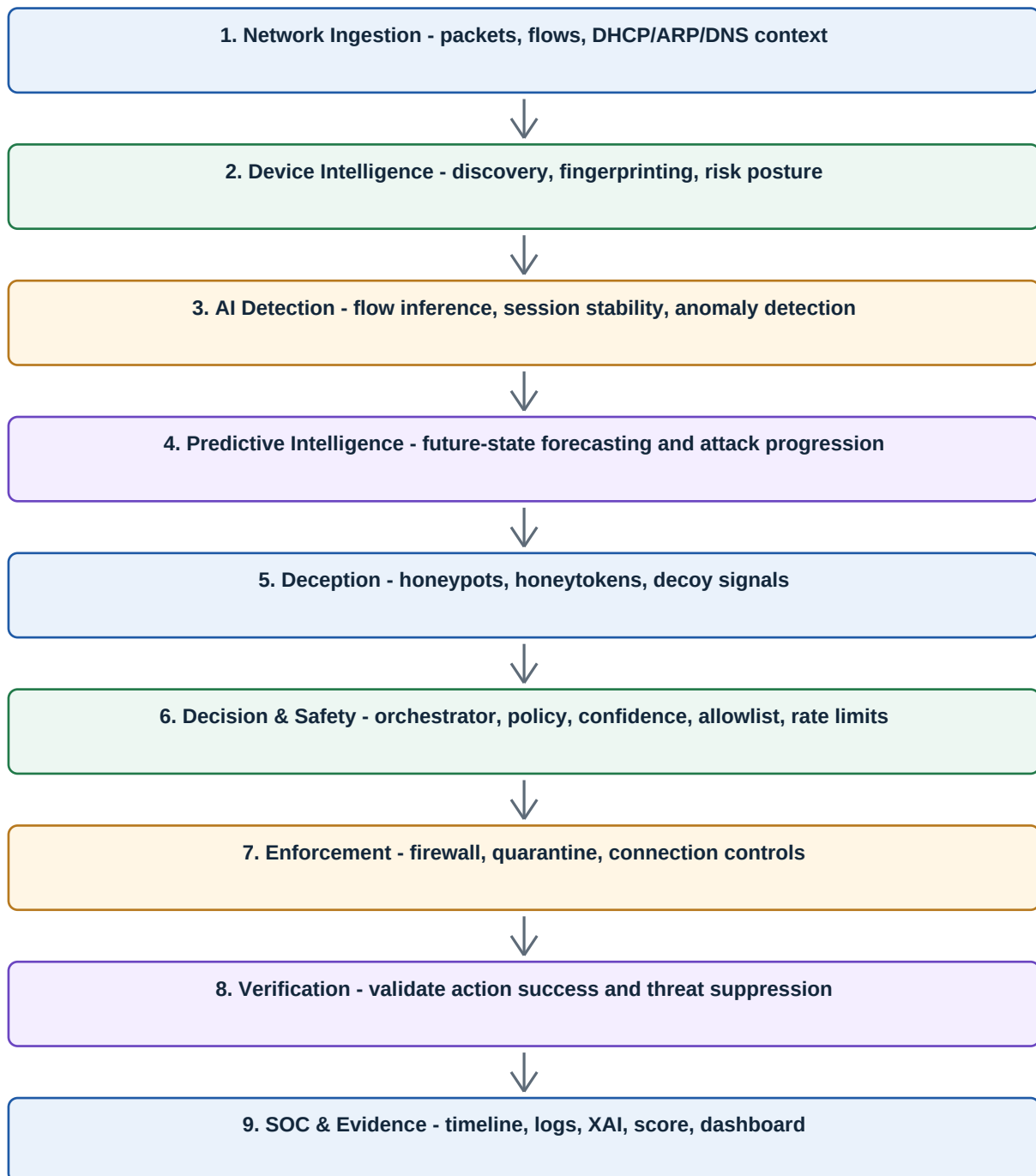
SYSTEM OBJECTIVE	PRIMARY IMPLEMENTATION DECISION
Protect a smart-home network with continuous visibility, AI detection, behavioral intelligence, deception, controlled autonomous response, and verification.	Deploy the existing AI Home Shield intelligence stack on a Raspberry Pi 5-class edge appliance with two network interfaces and a protected downstream LAN.

Architecture rule: The Raspberry Pi is not just an attached sensor. It is the edge security appliance; the laptop is the development/administration console. The system must remain useful locally even when cloud services are unavailable.

Prepared as the master technical reference for the AI Home Shield hardware MVP and its evolution into a production-oriented smart-home security appliance.

1. Architecture at a Glance

The complete system is organized into nine functional layers and one physical testbed. Each layer has a clear responsibility, interfaces, evidence, and control boundaries.



Golden path: Internet -> Router -> AI Home Shield -> Protected LAN -> IoT devices. Threat telemetry flows inward; authorized enforcement flows outward through a policy gate.

2. Physical / Cyber-Physical Architecture

The hardware MVP turns the existing software into a real network security appliance. The recommended topology places AI Home Shield inline so traffic can be observed and, when policy permits, blocked at the network edge.

PHYSICAL NETWORK TOPOLOGY

- Internet / ISP / ONT
- Home Router - Wi-Fi, DHCP, ordinary home connectivity
- AI Home Shield - Raspberry Pi 5-class edge appliance
- Upstream interface -> router; downstream interface -> protected LAN
- Protected devices - camera, smart TV, smart plug, bulb, sensor, ESP32, etc.

Why two interfaces? A dual-interface design supports true inline gateway operation. A single-interface design is acceptable for the first passive-monitoring experiment, but it is not the final enforcement topology.

Component	Role	MVP decision
Raspberry Pi 5 - 8GB class	Edge compute node	Required
USB 3 Gigabit Ethernet adapter	Second network interface	Required for inline MVP
Official / stable power supply	Reliable operation	Required
Active cooling	Sustained CPU/network workload	Required
High-endurance microSD	Boot + initial storage	Required
USB 3 SSD	Logs, baselines, evidence, datasets	Strongly recommended
Cat6 Ethernet	Upstream/downstream links	Required
RGB LED + buzzer	Physical security state feedback	Recommended for demo
OLED	Local status summary	Optional

3. Internal Software Architecture

The current repository already contains the primary agentic building blocks: discovery, risk profiling, PerceptionAgent, SessionAggregationAgent, BaselineAgent, deception, OrchestratorAgent, FirewallAgent, SOC timeline, XAI, and Home Shield Score. The hardware integration adds capture, flow construction, local storage, safety, verification, and platform abstraction.

EDGE SECURITY CORE

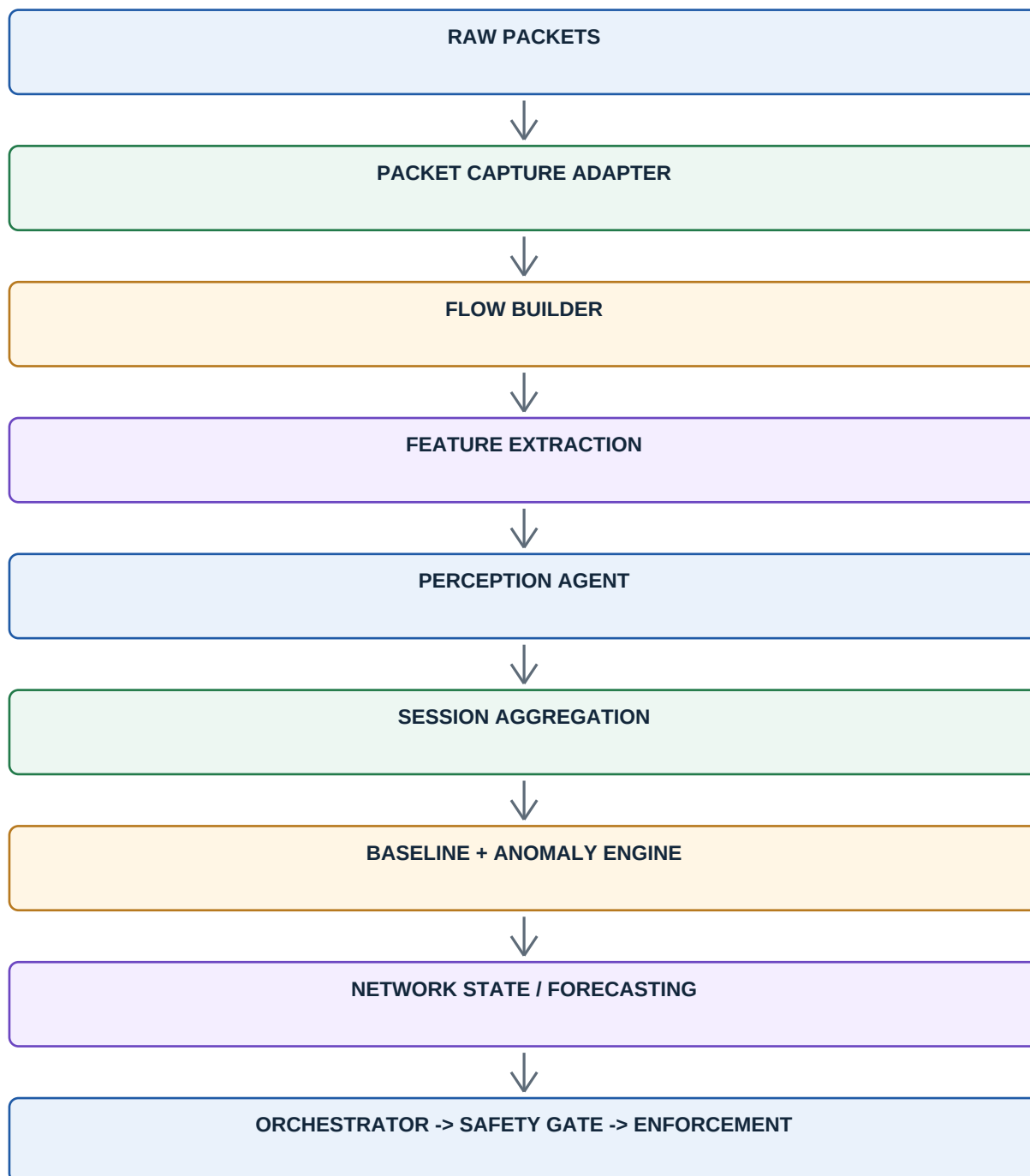
- Capture Adapter - tshark/libpcap or equivalent packet source
- Flow Builder - converts packets into bidirectional/time-windowed flows
- Feature Extractor - produces model-ready features
- Agent Runtime - discovery, risk, perception, session, baseline, prediction, deception
- Decision Plane - orchestrator plus policy/safety gate
- Enforcement Plane - Linux firewall/nftables/iptables abstraction
- Local State - SQLite + append-only logs + model store
- Management Plane - Streamlit initially; API/web UI later

Separation rule: Detection models may propose actions; only the orchestrator and policy gate may authorize high-impact actions. The model itself must never directly execute firewall commands.

Existing module / feature	Final hardware role	Status
Discovery Agent + Nmap	LAN discovery and device inventory	Reuse
Risk Agent	Device security posture	Reuse
PerceptionAgent	Per-flow threat inference	Reuse + live feature adapter
SessionAggregationAgent	Temporal stabilization	Reuse
BaselineAgent	Device/network normal behavior	Reuse
Honeypot / Honeytokens	High-confidence deception evidence	Reuse
OrchestratorAgent	Decision logic	Reuse + safety gate
FirewallAgent	Enforcement abstraction	Port to Linux backend
Streamlit UI	Local management/demo console	Reuse initially
Attack Demo	Controlled validation generator	Reuse / replace with lab traffic

4. Real-Time Network Telemetry Pipeline

The live pipeline must be built around flow/session semantics. Sending every raw packet directly into the ML model is not the target architecture. Packets are captured, aggregated, featurized, inferred, and correlated over time.



Feature group	Examples	Why it matters
Identity	src/dst IP, MAC, device ID	Maps activity to devices
Transport	protocol, src/dst port, TCP flags	Detects scans, protocol abuse, suspicious services
Volume	packets, bytes, duration	Captures floods and abnormal transfer
Timing	inter-arrival time, burstiness	Captures automation/beaconing
Behavioral	destination count, fan-out, port diversity	Captures reconnaissance and deviation
Session	persistence, flow count/window	Reduces single-flow noise

Feature group	Examples	Why it matters
Context	baseline deviation, device risk	Adds device-specific meaning

Live-vs-demo separation: The system may retain CICIoT replay and controlled attack generators for repeatability, but production architecture must treat the packet/flow capture interface as the authoritative live telemetry source.

5. AI Intelligence Architecture

The intelligence stack has two layers: a near-real-time detection path that is already present in the project, and a predictive path that extends the system toward attack forecasting and SIH-oriented research.

DETECTION PATH

- Per-flow probability + label
- Sliding-window session stability
- Device and network baselines
- Behavioral anomaly classes
- XAI evidence for alert explanation

PREDICTIVE PATH - TARGET EXTENSION

- Build a time-indexed network state representation
- Encode state sequences with a temporal model
- Forecast next network state(s) over K windows
- Map predicted behavior to attack stages / ATT&CK techniques where defensible
- Expose probability, horizon, evidence, and uncertainty
- Feed predictions to orchestration as one signal - never as an unconditional command

Signal	Question answered	Action influence
Perception score	Is this flow suspicious?	Alert / candidate action
Session score	Is the behavior persistent?	Raises confidence
Baseline deviation	Is this unusual for the device/network?	Raises or lowers confidence
Deception hit	Did an attacker interact with a decoy?	High-confidence corroboration
Forecast	Where is the behavior likely to progress next?	Prioritize preventive action
Device risk	How exposed is the affected asset?	Response severity weighting

6. Deception and High-Confidence Evidence

Deception is not a decorative feature. It is a second evidence channel that can corroborate model suspicion and reduce the chance of acting on a false positive.

DECEPTION LAYER

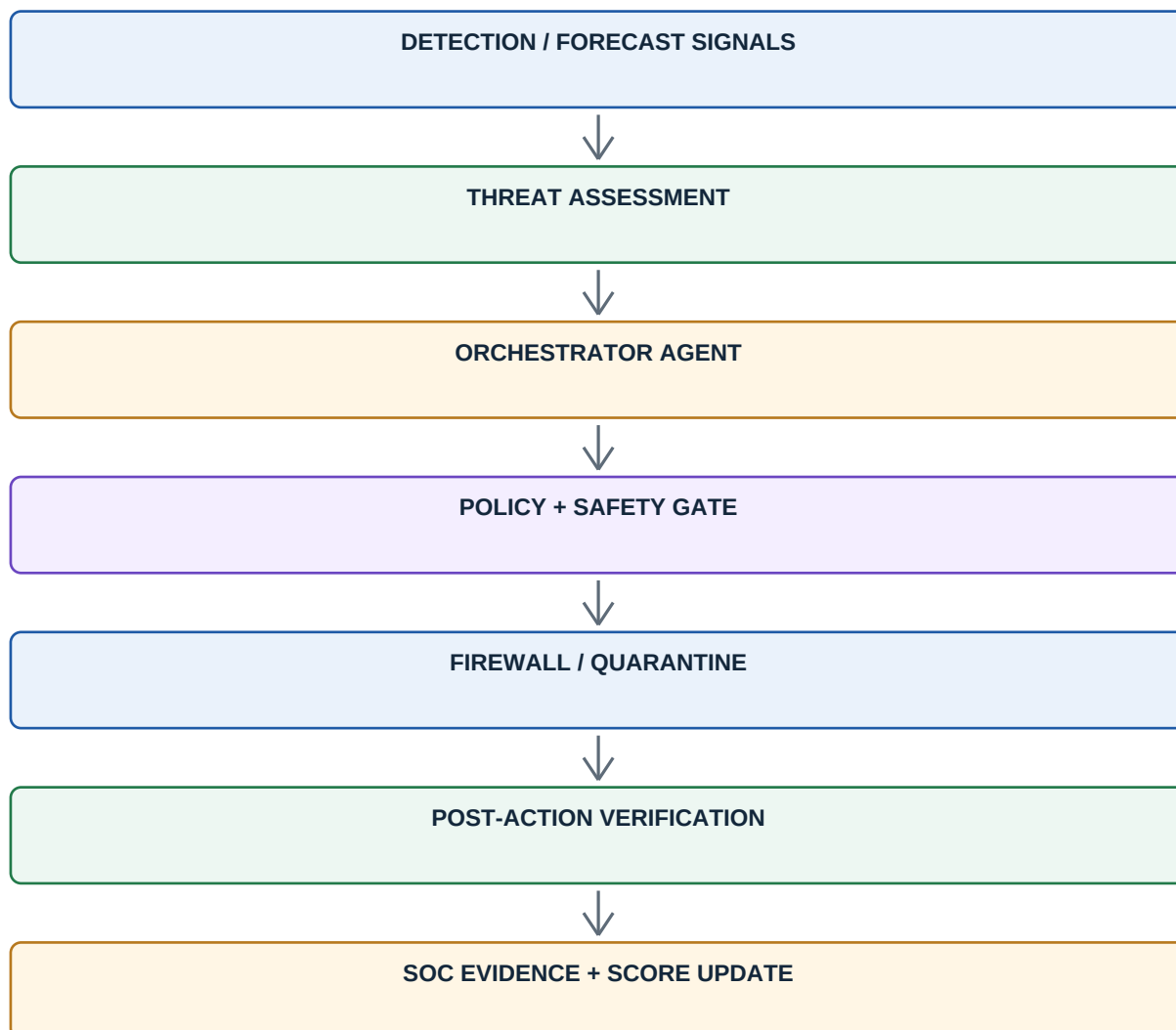
- Honeypot service - listens on controlled decoy port(s)
- Honeytokens - bait files with unique identifiers
- Optional future decoy services - only isolated and explicitly authorized
- Every trip produces a timestamped SOC event
- Autonomous mode may use deception signals to increase response confidence

Example: A camera shows a sudden destination fan-out and abnormal connection rate. The behavioral model raises suspicion. The same source then touches a honeytoken or honeypot. The orchestrator receives multiple correlated signals and can cross the response threshold.

Signal combination	Interpretation	Recommended decision
ML only	Possible threat	Alert / continue observation
ML + session persistence	Sustained suspicious behavior	Increase confidence
ML + baseline anomaly	Device-specific deviation	Increase confidence
ML + deception	Independent corroboration	Strong candidate for block
ML + baseline + deception	Multiple independent signals	Highest confidence; policy may auto-block

7. Decision, Safety and Autonomous Response

The autonomous loop is deliberately constrained. A model can recommend; the policy engine authorizes; the enforcement adapter executes; verification confirms the outcome.



Safety control	Purpose	Implementation target
Safe Mode	Choose automatic vs controlled/manual behavior	AUTO / ON / OFF semantics
Autonomous Mode	Enable closed-loop response	Explicit user toggle
Protected allowlist	Prevent critical trusted devices from blind blocking	Persistent local list
Confidence threshold	Require adequate evidence	Configurable by severity
Multi-signal confirmation	Require 2-of-3 or equivalent for high-impact actions	ML + baseline + deception/session
Action rate limiter	Prevent action storms	Per IP/device/time window
Dry-run mode	Test policies without enforcement	Mandatory during initial lab setup
Audit trail	Explain every decision	Timestamp + evidence + action + outcome
Verification	Check that block actually worked	Re-check flow / rule state / traffic

Non-negotiable boundary: High-impact actions must pass through a policy gate. The raw output of an AI model must never invoke a firewall command directly.

8. Enforcement and Platform Abstraction

The current code uses Windows Firewall/netsh for the laptop prototype. The hardware appliance should use a Linux-native enforcement backend behind the same abstract FirewallAgent interface so the rest of the product remains platform-independent.

Layer	Current prototype	Hardware MVP	Future product
Firewall API	FirewallAgent	FirewallAgent abstraction	Stable cross-platform policy API
Backend	Windows netsh	nftables/iptables backend	Hardened packet filter + connection tracking
Quarantine	Logical / controlled action	Dedicated LAN/VLAN/subnet strategy where feasible	Policy-based network isolation
Blocking target	IP	IP / device / flow tuple as supported	Device identity + policy group
Persistence	Local logs/state	SQLite + firewall state reconciliation	Signed config + durable state

RESPONSE SEQUENCE

- Receive approved action
- Validate target against allowlist and current state
- Apply firewall/quarantine rule
- Record command/result
- Verify traffic suppression
- Emit SOC event
- Update Home Shield Score and device state

9. Local Data, Evidence and Service Layout

The appliance should be local-first. Network telemetry remains on the device by default. Cloud services can be added later as optional enrichment, fleet management, or mobile notification layers.

Store / service	Data	Retention idea	Purpose
SQLite	devices, baselines, scores, actions	Configurable	Fast local state
Append-only event log	SOC events	Days/weeks	Audit trail
Firewall log	Rules, results, targets	Days/weeks	Enforcement evidence
Honeypot log	Connections / trips	Days/weeks	Deception evidence
Honeytoken log	Trips / token IDs	Days/weeks	High-confidence evidence
Model store	ML model files / metadata	Versioned	Inference
Optional SSD capture store	Flow archives / PCAP samples	Short rolling window	Debugging / research

Privacy principle: collect the minimum telemetry needed for detection and verification. Prefer local processing and provide transparent user controls for any future off-device data transfer.

SERVICE GROUPS ON PI

- network-capture.service
- flow-engine.service
- ai-home-shield.service
- deception.service
- firewall-agent.service
- evidence-store.service
- dashboard.service

10. Management / Dashboard Architecture

Streamlit remains the fastest way to demonstrate the current product. The final architecture treats the UI as a management plane, not as the security engine itself.

UI area	Primary information	Hardware integration
Overview	Home Shield Score, device count, threat count	Reads local API/state
Devices	Inventory, risk score, ports, anomalies	Reads discovery/risk services
Live Threat Monitor	Flows, sessions, probabilities, anomalies	Reads live event stream
Prediction	Current state, forecast, horizon, confidence	Reads forecasting service
Deception	Honeypot/honeytoken status and hits	Reads deception events
Response	Blocks, quarantines, action outcomes	Invokes policy-approved actions
SOC Timeline	Chronological evidence	Reads append-only event store
System Health	CPU, memory, event rate, service status	Reads agent/service health

Migration path: Streamlit -> local REST API -> modern local web UI/mobile companion. Do not perform a full UI rewrite before the hardware telemetry and enforcement loop is stable.

11. Repository-to-Architecture Mapping

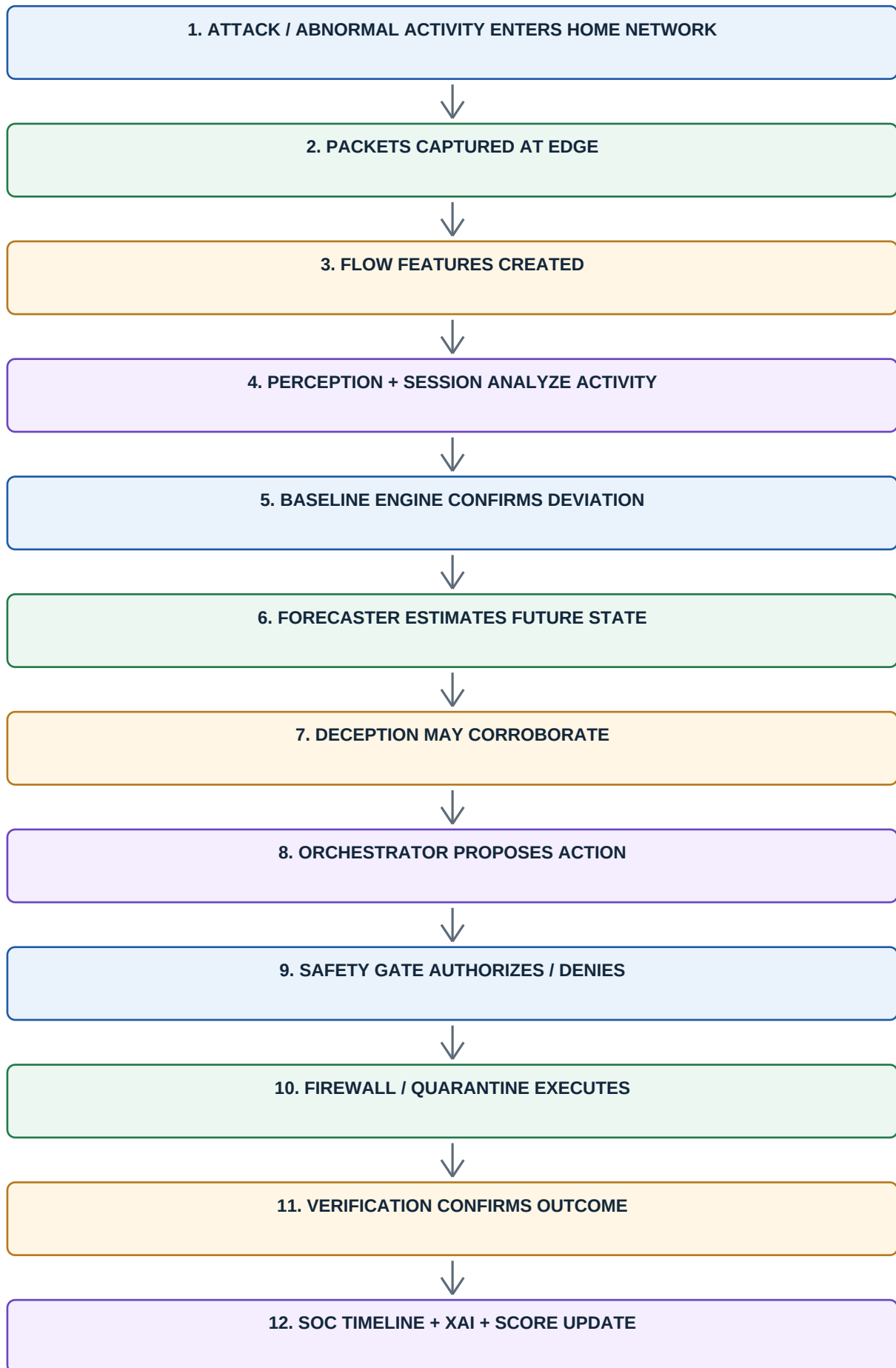
The implementation should evolve from the existing repository rather than becoming a second unrelated codebase.

Repository area	Final responsibility	Planned additions
app.py	Management UI / orchestration of views	Move security logic behind service/API boundaries over time
agents/	Security intelligence	network_state_agent.py, attack_forecasting_agent.py, safety_agent.py, verification_agent.py
utils/	Platform and demo utilities	packet/flow adapters, hardware helpers
models/	Model artifacts	versioned detection + forecasting models
data/	Sample/test data	controlled replay fixtures + schema definitions
logs/	Local evidence	rotation + structured event schema
hardware/	GPIO and network hardware abstraction	LED, buzzer, display, NIC discovery/status
capture/	Telemetry pipeline	packet_capture.py, flow_builder.py, feature_extractor.py
storage/	Local persistence	database.py, baselines.py, evidence.py
security/	Safety controls	policy.py, allowlist.py, audit.py

Architecture freeze rule: New modules should plug into the defined interfaces. Avoid putting hardware-specific code inside ML agents; avoid putting security decisions directly in the Streamlit UI.

12. End-to-End Threat Lifecycle

This is the canonical flow to implement, test, and narrate. Every transition should create measurable evidence.



13. Physical Demo / Judge Scenario

The demo should prove that the box is physically part of the network, not merely running a simulated dashboard.

Stage	Visible proof	Target message
1. Show hardware	Pi + two network links + testbed	This is the security appliance
2. Show topology	Router -> Shield -> IoT devices	All protected traffic passes through the edge
3. Discover devices	Camera/TV/plug/etc. appear	AI Home Shield understands the network
4. Establish baseline	Normal traffic and device posture	The system learns expected behavior
5. Start live monitoring	Live flows and sessions	Traffic is being processed in real time
6. Trigger controlled attack	Second laptop/VM/ESP32 generates test activity	The threat is real traffic in a controlled lab
7. Detection	Perception/session/baseline signals appear	AI recognizes the attack pattern
8. Forecast / explanation	Predicted state + evidence	System understands trajectory, not only a single packet
9. Deception	Honeypot/honeytoken signal	Independent evidence increases confidence
10. Autonomous response	Firewall rule + red status indicator	The box can act at the edge
11. Verification	Traffic stops / state changes	Response was actually enforced
12. SOC proof	Timeline + logs + score	Every action is auditable

14. Hardware Bill of Materials and Build Phases

Phase	Hardware	Purpose
Core appliance	Raspberry Pi 5 8GB class	Edge compute
Core appliance	USB 3 Gigabit Ethernet adapter	Second interface / inline topology
Core appliance	Stable USB-C power + active cooling	Reliability
Core appliance	128GB high-endurance microSD	Boot + initial storage
Core appliance	3 x Cat6 cables	Network links + bench setup
Performance upgrade	500GB USB 3 SSD	Logs, flow archives, models, research data
Demo upgrade	RGB LED + buzzer	Immediate physical alarm/status
Demo upgrade	OLED display	Local appliance status
Testbed	IP camera, smart plug, bulb, sensor, ESP32	Representative smart-home devices
Attack lab	Second laptop/VM or second SBC	Controlled traffic generation

Do not build custom hardware yet. Prove the cyber-physical loop first. Once the Raspberry Pi prototype is stable, the next hardware generation can move to an enclosure with native dual Ethernet, SSD, secure boot/secure element, signed updates, and a hardened power/network design.

15. Implementation Roadmap - From Existing Software to Final MVP

Phase	Deliverable	Exit criterion
P0 - Freeze	Repository, architecture, interface definitions	No conflicting service responsibilities
P1 - Edge setup	Pi OS + networking + second NIC + service bootstrap	Pi can route/bridge lab traffic reliably
P2 - Live capture	Packet capture + flow builder + local feature schema	Known test traffic becomes model-ready flows
P3 - Live inference	Perception + session + baseline on live flows	Live attack patterns appear in dashboard
P4 - Deception	Honeypot + honeytokens + evidence correlation	Trips appear with timestamps and source identity
P5 - Safety	Allowlist + thresholds + action rate limit + dry-run	Unsafe actions are blocked by policy
P6 - Enforcement	Linux firewall backend + verification	Approved block demonstrably stops traffic
P7 - Prediction	Network-state representation + forecasting model	System predicts future state on replay/test sequences
P8 - Product demo	Physical enclosure, LEDs, polished UI, scripted testbed	Repeatable 2-5 minute demo
P9 - Validation	False-positive analysis + latency + resilience + recovery tests	Measured evidence supports claims

Priority: P1-P6 first. Predictive forecasting (P7) is an extension after the live security loop works end-to-end. The appliance must protect and verify before it becomes a research forecasting platform.

16. Validation and Test Matrix

Test family	Example test	Expected result
Connectivity	Normal home traffic through appliance	No unacceptable disruption
Discovery	New IoT device joins LAN	Device appears in inventory
Port posture	Known open/closed ports	Risk posture updates
Anomaly	New outbound destination / port	Baseline anomaly generated
Attack detection	Controlled scan/flood/beacon pattern	Perception/session evidence rises
Deception	Connect to honeypot / access token	High-confidence SOC event
Safety	Attempt to block allowlisted IP	Policy denies action
Rate limiting	Trigger repeated actions	Excess actions suppressed
Firewall	Approved test source blocked	Traffic is actually denied
Verification	Remove rule / recover device	System returns to expected state
Failure handling	Capture process fails	Dashboard shows degraded state, no unsafe action
Resource	Sustained traffic load	System stays within defined CPU/memory/latency budget

Measure, do not assume: detection accuracy, false positives, false negatives, flow-to-alert latency, decision latency, enforcement latency, CPU/memory use, packet loss, and recovery time should be benchmarked on the hardware before making quantitative performance claims.

17. Security Model and Trust Boundaries

The security architecture itself must be defended. The appliance is privileged, therefore compromise of the management and enforcement plane would be high impact.

Trust zone	Contains	Key protection
Untrusted	Internet, unknown network traffic, attacker-controlled devices	Strict packet filtering and input validation
Telemetry zone	Raw packet/flow data	Bounded parsers, resource limits, schema validation
AI zone	Models, feature pipelines, agent runtime	Versioning, sandboxing where possible, least privilege
Decision zone	Orchestrator + policy gate	Explicit policy evaluation, allowlist, audit
Enforcement zone	Firewall and quarantine controls	Privileged service boundary, command validation
Evidence zone	Logs, database, model metadata	Append-only/auditable records, local access controls
Management zone	Dashboard/API	Authentication, local-only binding by default, secure session handling

Future hardening: secure boot, hardware-backed identity/keys, signed OTA updates, privileged service separation, tamper-evident logs, encrypted sensitive local state, and third-party penetration testing.

18. Future Product Architecture

The Raspberry Pi MVP validates the architecture. It is not necessarily the final commercial appliance. The production design should preserve the same logical planes and replace commodity development hardware with purpose-built, security-hardened infrastructure.

COMMERCIAL APPLIANCE - TARGET

- Purpose-built ARM/x86 compute module
- Native dual or multi-Gigabit Ethernet
- SSD storage
- Secure boot + TPM / secure element
- Hardware watchdog + resilient power
- Signed OTA update system
- Hardened enclosure
- Local-first management + optional fleet service
- Optional Wi-Fi radio only when its security model is explicit

MVP proof	Production evolution
Raspberry Pi 5 + USB NIC	Custom/industrial board with native multi-NIC
microSD / USB SSD	NVMe/eMMC/industrial SSD
Linux packages + local scripts	Signed services, secure boot, immutable/controlled update path
Streamlit	Hardened local web UI + API
Simple logs	Tamper-evident audit + telemetry pipeline
Manual hardware assembly	Certified enclosure, thermal/power design, manufacturing test

19. SIH / Research Alignment Strategy

The architecture should remain a real product first. The strongest SIH-oriented research extension is the predictive network-state layer: move from “detect this event” toward “forecast where the attack trajectory is going next,” then feed that prediction into policy-controlled prevention.

Research component	Current baseline	Target architecture
Traffic understanding	Flow + session inference	Time-indexed network-state representation
Anomaly detection	Device/network baselines	Temporal deviations across windows
Threat intelligence	Attack label/probability	Forecasted future state + confidence
Explainability	Why was attack detected?	Why this future state is predicted + strongest temporal evidence
Response	Autonomous firewall action	Predictive, policy-constrained prevention
Benchmarking	Replay / demo traffic	Open benchmark + controlled live testbed

Positioning statement: AI Home Shield integrates edge telemetry, behavioral baselining, AI threat detection, deception, attack forecasting, policy-controlled autonomous response, and verification into one smart-home security appliance architecture.

20. Master Definition of Done

Area	Definition of done
Physical	The appliance is visibly between the router and protected test devices in the lab topology.
Telemetry	Live packets become flow/session features without relying on dataset replay for the core path.
Detection	Perception + session + baseline logic work on live traffic and generate evidence.
Deception	Honeypot/honeytoken events correlate to source identity and produce auditable evidence.
Prediction	Forecasting module can estimate future network state on validated sequences and expose uncertainty.
Decision	Orchestrator routes all actions through policy/safety controls.
Enforcement	Approved action changes Linux firewall state and demonstrably suppresses the test threat.
Verification	The appliance confirms the action outcome and records success/failure.
Observability	SOC timeline, logs, XAI, score, and agent health remain available locally.
Safety	Allowlist, confidence threshold, rate limit, dry-run, audit trail, and safe mode are implemented before autonomous high-impact actions.
Performance	Latency and resource usage are measured on the actual hardware.
Recovery	Services can restart safely; corrupted/degraded subsystems fail closed or degrade gracefully without issuing uncontrolled actions.
Documentation	Architecture, interfaces, setup, test procedures, and threat model are maintained with the code.

Final architectural principle: AI Home Shield is a cyber-physical security appliance. The network edge provides real visibility and enforcement; the agentic software provides understanding and decisions; the safety layer controls autonomy; verification closes the loop; the SOC layer makes every action explainable and auditable.

Appendix A - Quick Reference

Item	Final decision
Core compute	Raspberry Pi 5 - 8GB class
Network topology	Router -> AI Home Shield -> Protected LAN
Interfaces	Two interfaces for inline MVP
Operating mode	Local-first edge appliance
Traffic path	Packet -> Flow -> Session -> AI -> Baseline -> Forecast -> Decision -> Enforcement -> Verification
Existing UI	Streamlit
Data store	SQLite + append-only logs; SSD recommended
Deception	Honeypot + honeytokens
Response	Firewall + quarantine
Safety	Safe Mode + Autonomous Mode + allowlist + confidence + 2-of-3 confirmation + rate limit + dry-run + audit
Research extension	Temporal attack forecasting / future-state prediction
Production direction	Secure boot, hardware-backed trust, native multi-NIC, SSD, signed OTA, hardened enclosure

This document is the architecture baseline to use when designing hardware, refactoring the repository, defining APIs, writing implementation prompts for coding agents, planning experiments, and preparing demos. Changes to the core topology or trust boundaries should be treated as architecture-level changes.

End of reference architecture.